OOP

UNIT 9 – EXCEPTION HANDLING

A clear interface must describe not only success, but also how failure is reported.

LEARNING OBJECTIVES

By the end of this unit, you should be able to:

- Explain what an exception is and when it is appropriate.
- Use throw, try, and catch in a complete C++ program.
- Trace both normal and exceptional execution paths.
- Catch standard exceptions by const reference and use what().
- Explain exception propagation and stack unwinding.
- Define a small custom exception type.
- Choose a catch location that can respond meaningfully.

FROM VALID OBJECTS TO FAILED OPERATIONS

Unit 8 ensured that every concrete media type implements play(). That compile-time guarantee does not mean that every call will succeed. A Song may be valid while its audio source is unavailable, and a user may request an index that does not exist.

A normal return value describes an ordinary outcome. An exception reports that a function cannot complete the operation promised by its interface. It keeps the failure separate from the normal result and lets a caller decide how to respond.

- throw reports the failure and interrupts the current normal path.
- try marks operations whose exceptions may be handled nearby.
- catch defines a handler for a compatible exception type.

The if statement still detects the bad condition. throw reports that condition to the caller. Detection and reporting are different jobs.

FIRST COMPLETE EXAMPLE: AN INVALID LIBRARY INDEX

MusicLibrary::getTitle checks whether an index identifies an existing title. An invalid index has no honest string result, so the function throws out_of_range. The handler in main prints the exception's message.

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class MusicLibrary {
private:
    static const int MAX_TITLES = 3;
    string titles[MAX_TITLES];
    int count;
public:
    MusicLibrary() : count(0) { }

    void addTitle(const string& title)    {
        if (count < MAX_TITLES)       titles[count++] = title;
    }
```

```
    string getTitle(int index) const
    {
        if (index < 0 || index >= count)
            throw out_of_range("Library index out of range");

        return titles[index];
    }
};

void printTitleAt(const MusicLibrary& library, int index) {
    string title = library.getTitle(index);
    cout << "Selected: " << title << endl;
}

int main() {
    MusicLibrary library;
    library.addTitle("Imagine");
    library.addTitle("Deep Focus");

    try
```

```
    {
        printTitleAt(library, 0);
        printTitleAt(library, 5);
        cout << "Both requests succeeded." << endl;
    }
    catch (const out_of_range& error)
    {
        cout << "Request failed: " << error.what() << endl;
    }

    cout << "Program continues." << endl;
}
```

The program prints:

```
Selected: Imagine
Request failed: Library index out of range
Program continues.
```

TWO PATHS THROUGH THE SAME CODE

NORMAL PATH

1. main enters the try block and requests index 0.
2. getTitle validates the index and returns Imagine.
3. printTitleAt prints the title and returns normally.
4. Execution continues with the next statement in the try block.

EXCEPTIONAL PATH

1. The next call requests index 5.
2. getTitle detects the invalid index and executes throw.
3. getTitle and printTitleAt end without completing normally.
4. The out_of_range handler in main runs.
5. Execution continues after the complete try-catch statement.

Both requests succeeded is skipped because it appears later in the interrupted try block. Execution does not return to the statement immediately after throw.

THE EXCEPTION OBJECT, WHAT(), AND STANDARD TYPES

throw out_of_range("Library index out of range") constructs an exception object. Its type describes the category of failure, and what() returns its explanatory message. Exception objects should normally be caught by const reference.

```
catch (const out_of_range& error)
{
    cout << error.what() << endl;
}
```

- invalid_argument: an argument has an unacceptable value.
- out_of_range: an index or position lies outside its valid range.
- runtime_error: an operation fails because of a run-time condition.

These classes derive from std::exception. Place a specific handler before a more general base-class handler because C++ enters the first compatible catch block.

EXCEPTION PROPAGATION

A function does not need to catch an exception merely because the exception passes through it. In the first example, getTitle throws, printTitleAt has no handler, and main catches the exception. This movement through unfinished calls is propagation.

C++ searches the active calls from the throwing point outward and stops at the first compatible handler. If no matching handler is found, the program terminates. Catch the exception at a level that can make a sensible decision.

STACK UNWINDING AND DESTRUCTORS

As an exception propagates out of a function, its local automatic objects leave scope. C++ destroys them in reverse construction order before continuing its search for a handler. This process is called stack unwinding.

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class PlaybackSession {
private:
    string title;
public:
    PlaybackSession(const string& title) : title(title)    {
        cout << "Opening session for " << title << endl;
    }
    ~PlaybackSession()    {
        cout << "Closing session for " << title << endl;
    }
};
```

```
void playSong(bool sourceAvailable) {
    PlaybackSession session("Imagine");

    if (!sourceAvailable)
        throw runtime_error("Audio source unavailable");

    cout << "Playing Imagine" << endl;
}

int main() {
    try
    {
        playSong(false);
    }
    catch (const runtime_error& error)
    {
        cout << "Playback failed: " << error.what() << endl;
    }
}
```

The program prints:

```
Opening session for Imagine
Closing session for Imagine
Playback failed: Audio source unavailable
```

Closing session appears before the handler's message because PlaybackSession is destroyed while playSong is being unwound. This is why cleanup should be tied to automatic objects and destructors.

A CUSTOM PLAYBACKERROR

A standard type is preferable when its meaning is sufficient. A custom type is useful when the program needs a domain-specific failure category. PlaybackError means that a media item exists but cannot complete playback.

```
class PlaybackError : public runtime_error
{
public:
    PlaybackError(const string& message)
        : runtime_error(message) { }
};
```

PlaybackError derives from runtime_error, so it inherits message storage and what(). It may be caught specifically as PlaybackError or generally as std::exception.

ADDING EXCEPTIONS TO THE MUSICLIBRARY

The MediaItem hierarchy from Unit 8 does not need to be repeated. Only the parts whose failure behavior changes are shown below. They belong to the same compiled MusicLibrary program.

SONG REPORTS AN UNAVAILABLE SOURCE

```
void play() const override
{
    if (!sourceAvailable)
        throw PlaybackError("audio source unavailable");

    cout << "Playing song: " << getTitle()
         << " by " << artist << endl;
}
```

Song detects the problem, but it does not decide whether the whole playlist stops.

MUSICLIBRARY REPORTS AND RECOVERS

```
const MediaItem& at(int index) const
{
    if (index < 0 || index >= count)
        throw out_of_range("Library index out of range");

    return *items[index];
}

void playAll() const
{
    for (int i = 0; i < count; ++i)
    {
        try { items[i]->play(); }
        catch (const PlaybackError& error)
        {
            cout << "Skipped " << items[i]->getTitle()
                 << ": " << error.what() << endl;
        }
    }
}
```

at() cannot return a valid MediaItem reference for an invalid index, so it throws out_of_range instead of inventing a special return value.

The handler is inside the loop. One PlaybackError therefore skips one item without cancelling playback of the remaining media items.

MAIN HANDLES AN APPLICATION-LEVEL SELECTION

```
try
{
    library.at(7).play();
}
catch (const out_of_range& error)
{
    cout << "Selection failed: " << error.what() << endl;
}
```

The program prints:

```
Playing song: Imagine by John Lennon
Skipped Missing Track: audio source unavailable
Playing podcast: Deep Focus with Ana Reyes
Selection failed: Library index out of range
```

WHERE TO CATCH AND WHEN TO RETURN

Throw near the place where a failure is detected. Catch near the place where a meaningful response is possible. Those locations are often different functions.

- Return a normal value when the caller routinely expects and checks that outcome.
- Throw when the function cannot provide its promised result and normal processing must stop.
- Catch only where the program can recover, report, translate, or terminate meaningfully.
- Do not catch an exception merely to hide it.

For example, add() may return false when a full fixed-capacity library is an expected condition. at() throws because it cannot return a valid reference for an invalid index.

COMMON BEGINNER MISTAKES

- Thinking throw replaces the condition check; if detects and throw reports.
- Throwing an int or string instead of a meaningful exception object.
- Catching exception objects by value instead of by const reference.
- Expecting execution to resume immediately after throw.
- Placing a general handler before a more specific handler.
- Using an empty catch block that silently hides a real failure.
- Using exceptions for ordinary outcomes that callers routinely test.
- Throwing from a destructor while another exception may already be propagating.

SUMMARY

An exception reports that an operation could not complete normally. throw reports the failure, try marks operations whose failures may be handled, and catch provides a response for a compatible exception type.

Exceptions may propagate through several function calls. Automatic objects are destroyed during stack unwinding. Use standard exception types when possible, define a custom type only when it adds meaning, and catch by const reference where the program can respond sensibly.

REVIEW QUESTIONS

1. What are the separate roles of throw, try, and catch?
2. What happens to the remaining statements in a try block after an exception is thrown?
3. Why should exception objects normally be caught by const reference?
4. What information does what() provide?
5. What is exception propagation?
6. What is stack unwinding, and why does PlaybackSession's destructor run?
7. Why should specific handlers appear before general handlers?
8. Why does playAll catch PlaybackError inside its loop?
9. When is a normal return value more appropriate than an exception?

LOOKING AHEAD

Exception handling adds a clear failure path to class interfaces. Every operation can now be designed with two questions in mind: what does it produce when it succeeds, and how does it report that it could not complete?